VAAN MEGAM NETWORKS · WIRELESS PROTOCOL DEVELOPMENT INTERNSHIP · TECHNICAL ASSESSMENT

TDMA Schedule Planner and Optimizer

Distance-2 graph colouring for collision-free time slots, proven optimal and verified on live packets in EMANE

9
slots for the 16-radio grid **proven optimal**

0
collisions in EMANE with the optimized schedule (5 runs)

396/396
link-level predictions matched by EMANE

102
automated tests, all passing

Part 1 · Network Brain	Complete: CLI, distance-2 colouring, heuristics, optimality proof, Slot × Node matrix
Part 2 · EMANE Engine	Implemented and measured: schedule → EMANE TDMA events → packets dropped or delivered per slot
Repository	Source, tests, this document, EMANE results and one-command demo
Date	October 2026

Contents

1. Summary
2. Understanding the problem
3. Design process: how the solution was reasoned out
4. Part 1: the Network Brain
5. Part 1 results: final values
6. Part 2: EMANE integration and results
7. Verification and quality
8. Design decisions and alternatives considered
9. Limitations and next steps
10. Appendix: running it, repository map, EMANE files

1. Summary

Radios that share one frequency take turns in time slots. Two radios must not share a slot if they are neighbours, or if they have a neighbour in common (the hidden terminal). I model those pairs as the square of the radio graph, **G^2**, so a collision-free schedule is exactly a proper colouring of G^2, with one colour per time slot.

- **Part 1.** A Python CLI builds the radio graph from coordinates (500 m range), builds G^2, and colours it with a portfolio of heuristics led by DSATUR. It then **proves** how good the result is: a lower bound from the largest group of mutually conflicting radios, plus a bounded exact search when needed. An independent validator re-checks every schedule before anything is printed.
- **Final value, assessment grid:** 9 slots, proven optimal, because 9 radios (`Node_06` and its 8 neighbours) all conflict with each other. On an irregular 16-radio layout the same code reaches 4 slots, so every slot is reused by 4 radios.
- **Part 2.** The schedule drives a 16-radio EMANE 1.5.2 TDMA network in one Docker container. Before each run the Brain predicts, for every link, whether a frame will get through. EMANE matched **396 of 396** predictions at 2–4 ms slots. The optimized schedule caused **zero collisions** in every run. A deliberately wrong control schedule (2-hop rule ignored) lost 97–99% of frames on exactly the links predicted.
- **One honest limit.** At the required 1 ms slots, the 2-vCPU test machine could not emulate 16 radios in real time, so some frames arrived after their slot ended. These were timing drops, never collisions; at 2 ms and 4 ms delivery was 98.7% and 99.7%.

2. Understanding the problem

2.1 Collisions happen at the receiver

In TDMA every radio gets an exclusive slot in a repeating frame. A collision is a *receiver* event: if two signals reach the same radio in the same slot, it decodes neither. That gives three cases:

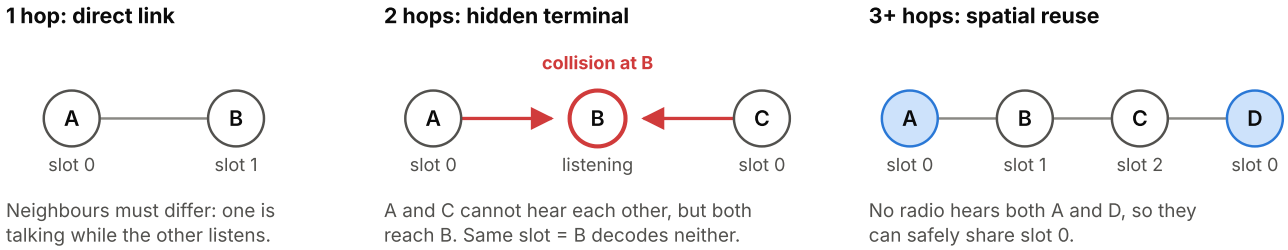


Figure 1. The three interference cases that define the scheduling rules.

2.2 From rules to a graph

G (communication graph): one node per radio, an edge when the Euclidean distance is ≤ 500 m (inclusive). **$H = G^2$** (conflict graph): an edge between two radios whose hop distance in G is 1 or 2. Those are precisely the pairs that must not share a slot.

> **Key insight.** A valid TDMA schedule is a proper vertex colouring of G^2. The minimum frame length is the chromatic number $\chi(G^2)$. Colouring G alone is *not* enough: in A–B–C, A and C are not adjacent in G, so they may get the same colour and collide at B.

2.3 Assumptions

Topic	Assumption	Reason
Links	Undirected; edge iff distance ≤ 500 m, compared exactly	As specified; no tolerance that would move the boundary
Interference	Protocol model: conflict iff 1 or 2 hops apart	As specified; standard in TDMA literature
Slots	One transmit slot per radio per frame (broadcast scheduling)	As specified; each node gets a slot
Topology	Static coordinates	As specified; mobility discussed in §9
Co-located radios	Distance 0 ⇒ linked, must differ	Radios in the same place certainly interfere
Separate groups	Disconnected components may reuse every slot	No radio can hear across components

3. Design process: how the solution was reasoned out

Each step below is a question I had to answer, the decision taken, and why. The order is the order in which the work was done.

1. **What exactly causes a collision?** → A receiver hearing two transmissions. → So the rule is "no shared slot within 2 hops", which is exactly G^2. This turns a radio problem into a well-studied graph problem.
2. **Can the minimum be found exactly?** → Minimum graph colouring is NP-hard in general, so no fast algorithm is known for every network. → Use fast heuristics, but do not just *claim* quality: measure it against a lower bound.
3. **Which heuristic?** → DSATUR was required as the baseline; I did not assume it is best. → I measured it against five other greedy orders, 100 seeded random orders and an iterated-greedy refinement on 290 random networks, and kept the best valid result. Ties keep DSATUR's answer, so extra attempts can never make the result worse.
4. **How do I know the answer is good?** → A clique (radios that all conflict pairwise) needs one slot per member, so its size is a hard minimum. → If the heuristic reaches it, the schedule is **proven optimal**. If not, a bounded branch-and-bound search proves or improves it, and otherwise the report says "best found".
5. **How do I know the code is correct?** → An independent validator recomputes 1- and 2-hop pairs by breadth-first search on G, without trusting G^2. The pipeline refuses to print an unverified schedule. 102 tests cover the 18 required cases and more, and deliberately injected bugs check that the tests catch real faults.
6. **Does it hold up on a real emulator?** → The Brain writes a full EMANE scenario and **predicts each link's outcome before the run**. A wrong "control" schedule runs alongside, proving the emulator would catch a bad schedule.
7. **What did I learn along the way?** → (a) For the assessment's own example grid, 9 slots is a proven minimum, so the 5-slot sample output in the brief cannot be collision-free (§5.2). (b) EMANE at 1 ms slots needs CPU headroom; slot length separates emulator timing from schedule correctness (§6.5).

4. Part 1: the Network Brain

4.1 Architecture

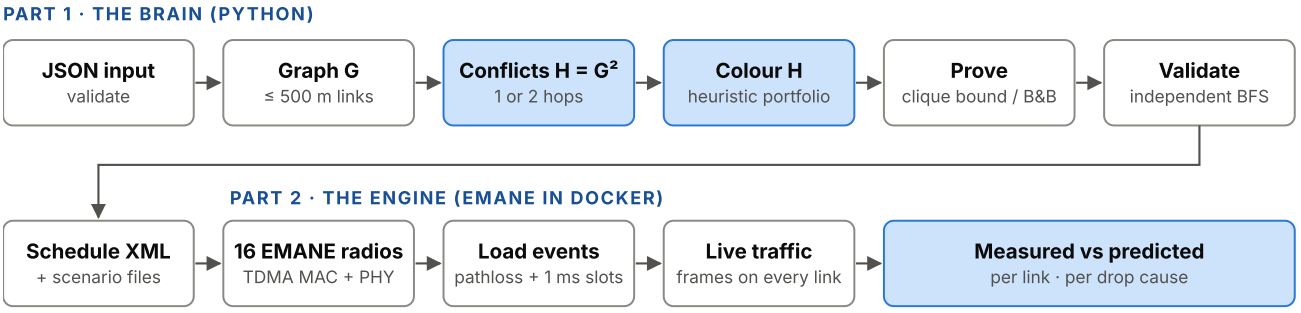


Figure 2. End-to-end pipeline. Blue boxes are the core of the method.

Module	Responsibility	Main library calls
`input_parser`	Strict JSON parsing: rejects malformed JSON, duplicates, strings, booleans, NaN/∞, wrong dimensions, wrong node count	`json.loads` (hooks), `math.isfinite`
`topology`	Builds G: every pair checked once, edge if distance ≤ range; components	`nx.Graph`, `math.dist`, `itertools.combinations`
`conflict_graph`	H = G², each edge tagged 1-hop or 2-hop	`nx.power(G, 2)`
`coloring`	Heuristic portfolio, keeps the best valid colouring, slots renumbered 0..K−1	`nx.coloring.greedy_color`, `random.Random(seed)`
`exact`	Clique lower bound; bounded branch-and-bound; optimality label	`nx.find_cliques`
`validator`	Independent check from G (not H); lists every violating pair	`nx.single_source_shortest_path_length`
`pipeline`	Runs the stages; raises instead of returning an invalid schedule	none
`report` / `cli`	ASCII report, mapping and matrix; flags, exit codes 0 / 2 / 3	`argparse`
`emane_export` / `emane_scenario`	Part 2 bridge: EMANE schedule XML, full scenario, per-link prediction	`xml.etree.ElementTree`

4.2 Colouring algorithm

Greedy colouring visits radios in some order and gives each the lowest slot not used by an already-coloured conflicting radio. It is always valid. Its quality depends only on the visiting order, so the method is about choosing good orders:

Attempt	Order rule	Why it is in the portfolio
DSATUR (baseline)	Next = radio whose neighbours already use the most distinct slots	Handles the most constrained radio first; strong and standard

Attempt	Order rule	Why it is in the portfolio
largest_first	Most neighbours first (Welsh–Powell)	Places hubs early
smallest_last	Peel least-connected radios, colour in reverse	Strong on sparse geometric graphs
independent_set, BFS, DFS	Structural orders	Diversity: different failure modes
random order ×100	Seeded random permutations (`random.Random(42)`)	Escapes blind spots of fixed rules; reproducible
iterated greedy	Re-colour keeping each slot's radios together	Can never use more slots, often fewer

4.3 Proving optimality

A **clique** in G^2 is a set of radios that all conflict pairwise, so each needs its own slot. Its size ω is therefore a lower bound: $\chi(G^2) \geq \omega$. In G^2 every radio together with all its neighbours forms a clique, so $\omega \geq$ (max degree + 1). The maximum clique is found exactly (Bron–Kerbosch). If the heuristic uses ω slots, the schedule is **proven optimal** and the clique is the certificate. If a gap remains, a **bounded branch-and-bound** search (DSATUR order, clique pre-coloured to break symmetry, 200,000-node budget) either finds a better schedule, proves the current one optimal, or stops and labels the result "best found". Example: 7 radios on a ring give $\omega = 3$, yet every heuristic needs 4. The search proves 4 is optimal in 4 search nodes.

4.4 Correctness safeguards

- **Independent validator**: recomputes all pairs within 2 hops by breadth-first search on G. A bug in building G^2 cannot validate itself.
- **Validation gate**: the pipeline raises `ScheduleValidationError` (exit code 3) rather than return an invalid schedule. A test injects a broken optimiser to prove this.
- **Determinism**: natural node order (`Node_2` before `Node_10`) and one seeded generator, so the same input and seed give byte-identical output.

5. Part 1 results: final values

5.1 The 16-radio assessment grid

Input: the assessment's coordinates, a 4×4 grid with 300 m spacing, `Node_01` at (0, 0) to `Node_16` at (900, 900). At 500 m range the diagonal neighbours (424 m) are linked too.

Measure	Value
Radios / links in G	16 / 42
Conflicting pairs in G^2	90 = 42 one-hop + 48 two-hop (hidden terminal)
Frame length	**9 slots** = 9 ms at 1 ms per slot
Lower bound (largest clique)	9: 01, 02, 03, 05, 06, 07, 09, 10, 11
Optimality	proven optimal: equals clique lower bound
Spatial reuse	5 of 9 slots shared; slot 0 used by 4 radios (01, 04, 13, 16)
Selected method	saturation_largest_first (DSATUR)
Independent validation	passed: 90 one/two-hop pairs checked

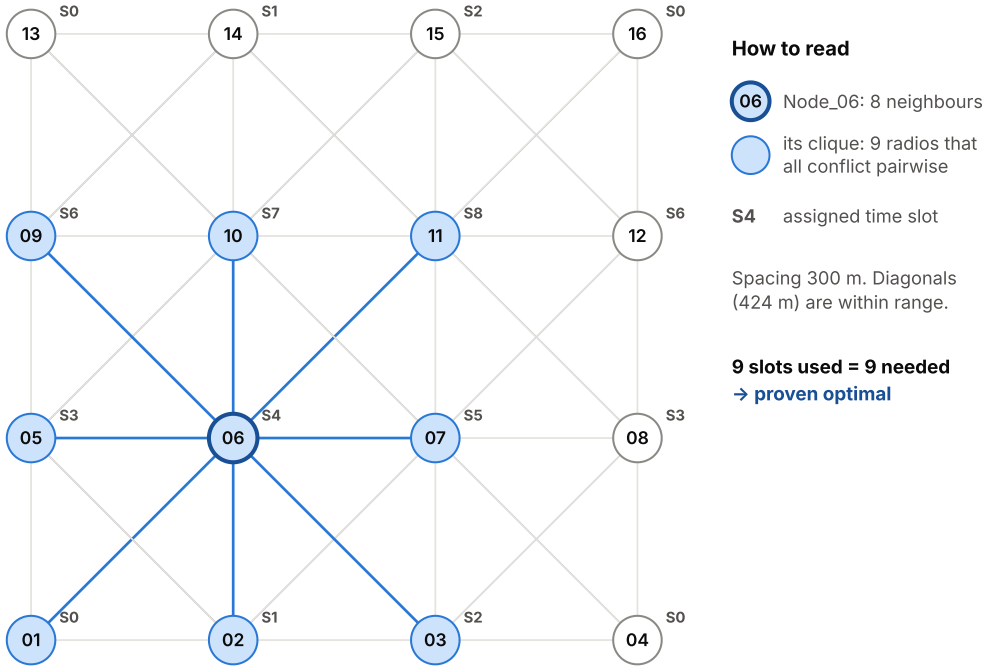


Figure 3. The optimal schedule on the assessment grid. The 9 highlighted radios all conflict pairwise, so no schedule can use fewer than 9 slots.

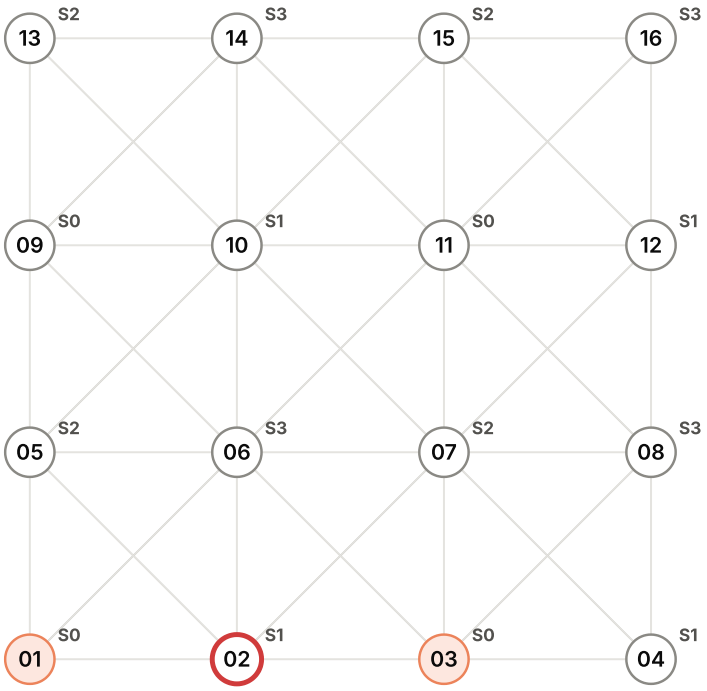
CLI output (unedited, from `tdma-optimizer --input examples/grid_16.json`):

```
NODE -> SLOT ASSIGNMENTS:
  Node_01 : Slot 0
  Node_02 : Slot 1
  Node_03 : Slot 2
  Node_04 : Slot 0
  Node_05 : Slot 3
  Node_06 : Slot 4
  Node_07 : Slot 5
  Node_08 : Slot 3
  Node_09 : Slot 6
  Node_10 : Slot 7
  Node_11 : Slot 8
  Node_12 : Slot 6
  Node_13 : Slot 0
  Node_14 : Slot 1
  Node_15 : Slot 2
  Node_16 : Slot 0

STRUCTURAL TDMA SCHEDULE MATRIX (Slot x Node Boolean Matrix):
(column headers omit the common prefix 'Node_')
Slot \ Node | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16
-----------------------------------------------------------------------------------------
Slot 00     | 1  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 1
Slot 01     | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0
Slot 02     | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0
Slot 03     | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0
Slot 04     | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0
Slot 05     | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0
Slot 06     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 1  | 0  | 0  | 0  | 0
Slot 07     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0
Slot 08     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0
-----------------------------------------------------------------------------------------
```

5.2 A note on the 5-slot sample output in the brief

The brief's sample shows 5 slots for these coordinates. Figure 3 proves at least 9 are needed. The sample also gives `Node_01` and `Node_03` the same slot although both neighbour `Node_02`, a hidden-terminal collision. I read the sample as an illustration of the output *format*. My validator rejects that assignment (a test shows which pairs collide), and Part 2 shows a 4-slot schedule of the same kind losing packets in EMANE.



Control schedule: DSATUR on G only. 4 slots. Every pair of neighbours differs, so it looks valid...

...but Node_01 and Node_03 both use slot 0 and both reach Node_02. Node_02 hears them overlap and decodes neither.

The Brain predicts 64 such failed links; EMANE confirms all 64 (§6).

Figure 4. Colouring G instead of G^2 gives fewer slots but lets hidden terminals collide.

5.3 Other topologies

Input	Description	Radios	Links	Conflicts (1-hop / 2-hop)	Groups	Slots	Bound	Status
`grid_16`	4×4 grid, 300 m spacing (assessment coordinates)	16	42	90 (42 / 48)	1	**9**	9	proven: equals clique lower bound
`field_16`	irregular convoy + separate outpost, 2 groups	16	15	30 (15 / 15)	2	**4**	4	proven: equals clique lower bound
`ring_7`	7 radios on a ring, 450 m apart	7	7	14 (7 / 7)	1	**4**	3	proven: exhaustive branch-and-bound
`spec_4`	4-node example from the problem statement	4	5	6 (5 / 1)	1	**4**	4	proven: equals clique lower bound

5.4 Heuristic comparison

Measured with `scripts/benchmark.py` (seed 7) on 200 random 16-radio deployments in a 1.5 km square. DSATUR alone reached the proven optimum every time, and the full portfolio was proven optimal in 200 of 200 networks (mean 7.67 slots, about 8 ms per network).

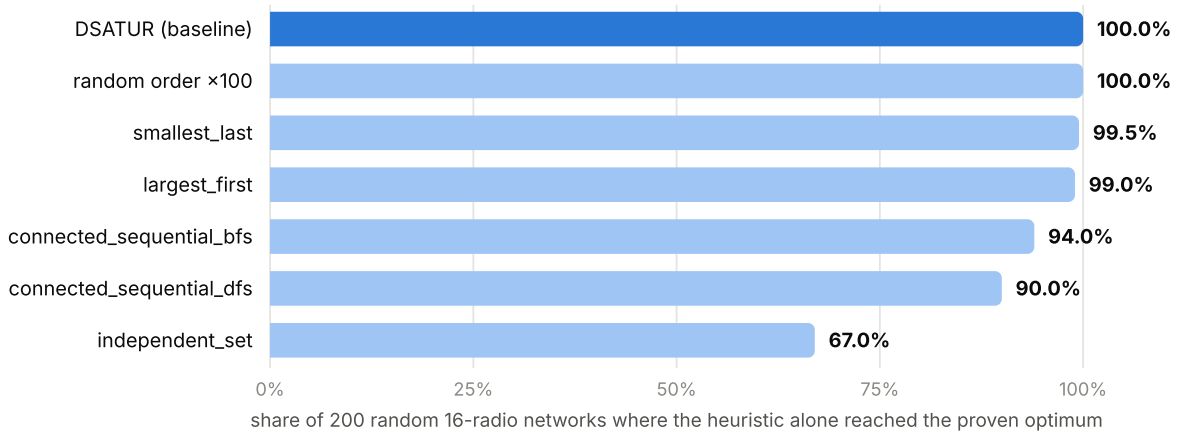


Figure 5. How often each heuristic alone reached the optimum. Weaker orders stay in the portfolio only for diversity; they can never worsen the result.

Network size	Trials	DSATUR optimal	Portfolio proven optimal	Median time
16 radios, 1.5 km square	200	200/200	200/200	8.4 ms
40 radios, 2.5 km	30	30/30	30/30	24.9 ms
80 radios, 3.5 km	30	29/30	30/30 (others caught DSATUR's miss)	59.8 ms
150 radios, 5 km	30	30/30	30/30	136.9 ms

6. Part 2: EMANE integration and results

6.1 What EMANE does, in plain words

EMANE emulates radios in real time. Each radio is an **NEM**: an `emane` process with a virtual network interface (`emane1` … `emane16`), a MAC layer and a PHY layer. A frame written into `emane5` is queued by NEM 5's MAC, sent over an emulated channel, and judged by every other NEM's PHY (signal strength, noise, overlap). If accepted, it comes out of their interface. EMANE's **TDMA radio model** lets a radio transmit only in its scheduled slots and receive in the others, so EMANE is the right tool to test whether the schedule works on packets.

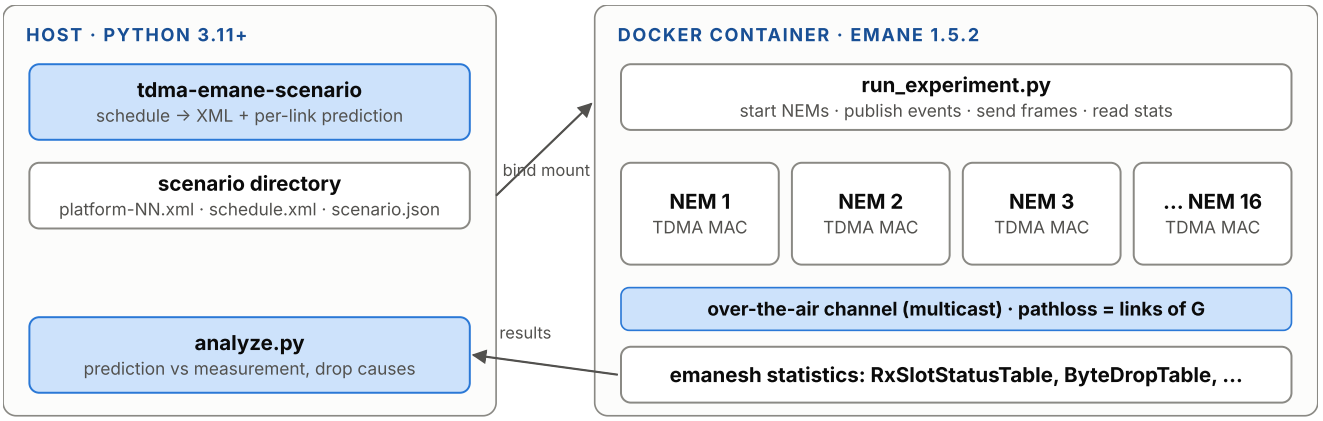


Figure 6. The host generates the scenario; one privileged container runs 16 NEMs.

Bridge file	Role
`emane_export.py`	Schedule → EMANE *full* TDMA schedule XML: `<structure slots=K slotduration=1000>`, one `<slot nodes=…><tx/>` per slot
`emane_scenario.py`	16 platform files, NEM/MAC/transport XML, the schedule, and `scenario.json` with the predicted outcome of every link
`emane/run_experiment.py`	In the container: starts NEMs, publishes pathloss and schedule events, sends frames, saves EMANE statistics
`emane/analyze.py`	Prediction vs measurement per link; splits drops into collisions and late frames
`emane/run_demo.sh`	All of the above in one command

6.2 Configuration

Setting	Value	Why
Slot / frame	1 ms slots (also 2 and 4 ms), 1 frame of K slots	1 ms as required; longer slots isolate emulator timing (§6.5)
Schedule type	Full schedule	Every unlisted slot becomes a receive slot, so each radio listens in the other K−1 slots
Channel	2.4 GHz, 1 MHz bandwidth, one frequency	The Brain assumes one shared channel
Data rate	11 Mbps	Its PCR curve needs ≥ 5 dB SINR, so equal-power overlaps (≈ 0 dB) fail, like a real radio. 1 Mbps would still decode and hide collisions

Setting	Value	Why
Propagation	Precomputed pathloss: 80 dB on each link of G, none otherwise	EMANE's 'who hears whom' becomes identical to G, so the test isolates the schedule
Traffic	Each radio broadcasts an 88-byte frame every 100 ms for 20 s	200 numbered frames per radio; every receiver counts per sender

6.3 Experiment design

Prediction (computed before each run): a frame on link u → v is delivered if and only if v is not transmitting in u's slot, and no other neighbour of v transmits in u's slot. A link **matches** when predicted-delivered links measure ≥ 90% delivery and predicted-lost links ≤ 10%. Two schedules run through the same emulator:

- **optimized**: the Part 1 schedule (9 slots on the grid). Predicts every link delivered.
- **G-only control**: DSATUR on G instead of G^2 (4 slots, like the brief's sample). Predicts 64 of 84 directed links lost. Its job is to prove the emulator would catch a bad schedule.

6.4 Results

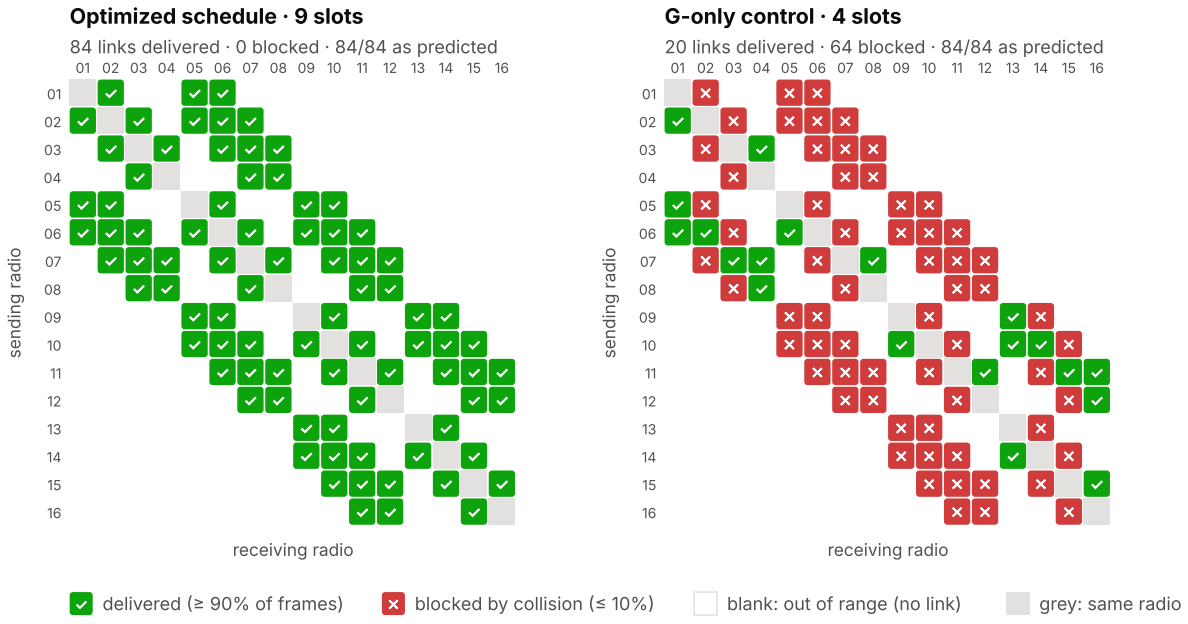


Figure 7. Measured delivery for every directed link of the grid, 4 ms slots. Every red cell was predicted lost before the run; every green cell was predicted delivered.

Topology	Schedule	Slots	Slot	Delivery: predicted-OK links	Delivery: predicted-lost links	Matches	Collision drops	Late drops
4×4 grid	**optimized**	9	1ms	79.7%	–	34/84	**0**	3,408
4×4 grid	G-only control	4	1ms	31.5%	16.6%	28/84	2,059	11,363
4×4 grid	**optimized**	9	2ms	98.7%	–	84/84	**0**	221
4×4 grid	G-only control	4	2ms	96.5%	2.9%	84/84	11,994	578
4×4 grid	**optimized**	9	4ms	99.7%	–	84/84	**0**	45

Topology	Schedule	Slots	Slot	Delivery: predicted-OK links	Delivery: predicted-lost links	Matches	Collision drops	Late drops
4×4 grid	G-only control	4	4ms	99.8%	1.1%	84/84	12,629	35
field	**optimized**	4	1ms	41.4%	–	0/30	**0**	3,514
field	G-only control	3	1ms	32.7%	5.9%	16/30	207	4,848
field	**optimized**	4	4ms	99.7%	–	30/30	**0**	17
field	G-only control	3	4ms	100.0%	1.2%	30/30	3,746	11

Collision drops: frames EMANE's TDMA MAC discarded because they overlapped another transmission at the receiver. Late drops: frames that reached the receiver after their slot ended (EMANE "Slot Error"). Both are counted in frames from EMANE's own statistics tables.

6.5 What the results show

1. **EMANE enforces the schedule exactly as predicted.** At 2 and 4 ms, 396 of 396 directed-link predictions matched (both schedules, both topologies). No frame ever crossed between radios out of range.
2. **The distance-2 schedule is collision-free on real packets.** Zero collision drops in all 5 optimized runs, including at 1 ms.
3. **The 2-hop rule is necessary.** The G-only control lost 97–99% of frames on exactly the predicted hidden-terminal links, with over 12,000 collision drops on the grid.

6.6 The 1 ms finding

At 1 ms, many frames in *both* schedules arrived after their slot had ended. 16 real-time EMANE processes on a 2-vCPU machine could not process every frame within 1 ms. That this is emulator timing, not the schedule, is shown by the data: for the same schedule, late drops fall from 3,408 to 221 to 45 frames as the slot grows from 1 to 2 to 4 ms, while collisions stay at 0. A host with more cores should close the gap at 1 ms; I have not measured that yet (§9).

7. Verification and quality

Area	Evidence
Automated tests	102 tests, all passing: parser (19), topology (9), conflict graph (10), colouring (14), exact search (10), validator (7), report (7), EMANE export (5), EMANE scenario (8), CLI end-to-end (13)
Required cases	All 18 cases from the brief map to named tests (exact 500 m boundary, 2-hop conflict, 3-hop reuse, disconnected groups, duplicates, no-edge input, reproducibility, 16-node end-to-end, …)
Cross-checks	`nx.power(G, 2)` equals an independent BFS on 25 random topologies; prediction agrees with the validator
Mutation check	9 bugs injected one at a time (exclusive boundary, G instead of G^2, validator ignoring 2 hops, …). 7 caught; the 2 that survive only affect speed, never the output
Reproducibility	Same seed ⇒ byte-identical report (tested); fresh-clone install verified; EMANE demo is one command
Honesty in output	Every schedule is labelled proven or best-found; invalid schedules never print

8. Design decisions and alternatives considered

The theme behind most choices: **at 16 radios, a provably correct and explainable answer is worth more than raw speed.**

Decision	Chosen	Alternatives	Why the choice fits
What to colour	G^2 (1 and 2 hops)	G only	G misses hidden terminals; the EMANE control run shows the cost
Baseline	DSATUR	Welsh–Powell, smallest-last	Required, and measured best (200/200 optimal)
Search strategy	Greedy portfolio + refinement	Tabu search, simulated annealing, genetic	Already proven optimal here in milliseconds; metaheuristics add complexity without a proof
Optimality	Clique bound + bounded branch-and-bound	ILP or SAT solvers (OR-Tools, PuLP)	Proof without heavy dependencies the brief asked to avoid
Machine learning	Not used	Neural graph colouring	Excluded by the brief; no validity guarantee; not explainable
Verification	Independent BFS validator	Check against G^2 itself	A checker must not trust what it checks
Graph library	NetworkX	igraph, own code	Required; readable and tested; speed is irrelevant at this size
Edge construction	All pairs, $O(n^2)$	KD-tree / spatial grid	120 pairs for 16 radios; worth it only at thousands
EMANE links	Precomputed pathloss from G	Location + free-space model	Makes the emulator's topology identical to G, so results isolate the schedule
EMANE data rate	11 Mbps	1 Mbps (tutorial default)	1 Mbps decodes at 0 dB SINR and would hide collisions

9. Limitations and next steps

Limitation	Next step
Protocol interference model (hard 500 m disk, 2-hop rule)	Distance-based pathloss and a carrier-sense range larger than decode range; only how G^2 is built changes
1 ms slots in EMANE need CPU headroom (late drops on 2 vCPUs)	Repeat at 1 ms on 8+ cores, or spread NEMs over several containers
Static topology	Recompute on change; send EMANE *update* schedules only for radios whose slot changed
One slot per radio, regardless of traffic	Demand-weighted slots: split a busy relay into d mutually conflicting copies
Broadcast traffic only in EMANE tests	Unicast and routed traffic (e.g. OLSR) over the same schedule
Exact search is bounded	Report already says 'best found' when the budget ends; an ILP backend could be optional

10. Appendix: running it, repository map, EMANE files

10.1 Run

```
# install (Python 3.11+)
python -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]"

# Part 1: file or inline JSON
tdma-optimizer --input examples/grid_16.json --range 500 --expected-nodes 16 --seed 42 --restarts 100
tdma-optimizer --coordinates '{"Node_01":[0,0],"Node_02":[300,0],"Node_03":[600,0],"Node_04":[300,40
0]}' --expected-nodes 4

# tests and benchmark
python -m pytest
python scripts/benchmark.py

# Part 2 (Docker; on Arch: sudo pacman -S docker && sudo systemctl enable --now docker)
emane/run_demo.sh                              # grid_16 at 1 ms and 4 ms, about 5 minutes
SLOTS_US="1000 2000 4000" emane/run_demo.sh examples/field_16.json
```

10.2 Repository map

Path	Contents
`src/tdma_optimizer/`	Part 1 package and the Part 2 bridge (one module per pipeline stage)
`tests/`	102 tests, one file per module plus end-to-end CLI tests
`emane/`	`run_demo.sh`, `run_experiment.py`, `analyze.py`, `Dockerfile`, `results/` (all measured data)
`examples/`	`grid_16`, `field_16`, `ring_7`, `spec_4` topologies
`docs/`	This PDF, design notes (algorithm, architecture, EMANE, test report), unedited sample outputs
`scripts/benchmark.py`	Heuristic comparison on random networks

10.3 EMANE schedule produced for the grid (excerpt)

```
<?xml version="1.0" encoding="UTF-8"?>
<emane-tdma-schedule>
  <structure frames="1" slots="9" slotoverhead="0" slotduration="1000" bandwidth="1M" />
  <multiframe frequency="2.4G" power="0" class="0" datarate="11M">
    <frame index="0">
      <slot index="0" nodes="1,4,13,16">
        <tx />
      </slot>
      <slot index="1" nodes="2,14">
        <tx />
      </slot>
      <slot index="2" nodes="3,15">
        <tx />
      </slot>
      <slot index="3" nodes="5,8">
        <tx />
      </slot>
      <slot index="4" nodes="6">
        <tx />
      </slot>
      <slot index="5" nodes="7">
        <tx />
      </slot>
      <slot index="6" nodes="9,12">
        <tx />
      </slot>
      <slot index="7" nodes="10">
        <tx />
      </slot>
      <slot index="8" nodes="11">
        <tx />
      </slot>
    </frame>
  </multiframe>
</emane-tdma-schedule>
…
```